

torch.nn.functional.conv3d#

<https://pytorch.org/docs/stable/generated/torch.nn.functional.conv3d.html>

Description:

Applies a 3D convolution over an input image composed of several input planes. This operator supports TensorFloat32. See Conv3d for details and output shape. In some circumstances when given tensors on a CUDA device and using CuDNN, this operator may select a nondeterministic algorithm to increase performance. If this is undesirable, you can try to make the operation deterministic (potentially at a performance cost) by setting `torch.backends.cudnn.deterministic = True`. See Reproducibility for more information.

Function Signature

```
torch.nn.functional.conv3d(input, weight, bias=None,  
stride=1, padding=0, dilation=1, groups=1) → Tensor#
```

Parameters

Code Examples

```
>>> filters = torch.randn(33, 16, 3, 3, 3)
>>> inputs = torch.randn(20, 16, 50, 10, 20)
>>> F.conv3d(inputs, filters)
```

Notes & Warnings

NOTE

Note In some circumstances when given tensors on a CUDA device and using CuDNN, this operator may select a nondeterministic algorithm to increase performance. If this is undesirable, you can try to make the operation deterministic (potentially at a performance cost) by setting `torch.backends.cudnn.deterministic = True`. See [Reproducibility](#) for more information.

NOTE

Note This operator supports complex data types i.e. `complex32`, `complex64`, `complex128`.

⚠ WARNING

Warning For `padding='same'`, if the weight is even-length and dilation is odd in any dimension, a full `pad()` operation may be needed internally. Lowering performance.

Detailed Documentation

Documentation

Applies a 3D convolution over an input image composed of several input planes.

This operator supports TensorFloat32.

See Conv3d for details and output shape.

In some circumstances when given tensors on a CUDA device and using CuDNN, this operator may select a nondeterministic algorithm to increase performance. If this is undesirable, you can try to make the operation deterministic (potentially at a performance cost) by setting `torch.backends.cudnn.deterministic = True`. See [Reproducibility](#) for more information.

This operator supports complex data types i.e. complex32, complex64, complex128.

`input` – input tensor of shape $(\text{minibatch}, \text{in_channels}, iT, iH, iW)$ $(\text{minibatch}, \text{in_channels}, iT, iH, iW)$

`weight` – filters of shape $(\text{out_channels}, \text{in_channels} / \text{groups}, kT, kH, kW)$ $(\text{out_channels}, \frac{\text{in_channels}}{\text{groups}}, kT, kH, kW)$ $(\text{out_channels}, \text{groups} / \text{in_channels}, kT, kH, kW)$

`bias` – optional bias tensor of shape (out_channels) (out_channels) .
Default: None

`stride` – the stride of the convolving kernel. Can be a single number or a tuple (sT, sH, sW) . Default: 1

implicit paddings on both sides of the input. Can be a string {'valid', 'same'}, single number or a tuple (padT, padH, padW). Default: 0 padding='valid' is the same as no padding. padding='same' pads the input so the output has the same shape as the input. However, this mode doesn't support any stride values other than 1.

For padding='same', if the weight is even-length and dilation is odd in any dimension, a full pad() operation may be needed internally. Lowering performance.

dilation – the spacing between kernel elements. Can be a single number or a tuple (dT, dH, dW). Default: 1

groups – split input into groups, `in_channels\text{in\_channels}` should be divisible by the number of groups. Default: 1